PIPELINE DE DATOS DE VENTAS GLOBALES

Integración, limpieza y transformación de datos

Ana María Tique Manrique – 2620261035

Juan Diego Londoño – 2620261053

Yaritxa Duarte Urueña – 2620261034

Andrés Felipe Benitez Barrios – 2620261033

Universidad de Ibagué

Fundamentos de Integración y Transformación de Datos

Natalia Millán

9 de noviembre de 2026

Contenido

Resumen....3
Introducción....4
Objetivo....5
Metodología....6
Contexto....7
Actividades a desarrollar....8
A. Diagrama de flujo del pipeline de datos....8
B. Identificar las fuentes de datos....8
C. Diagnosticar la calidad de los datos....9
D. Aplicar las transformaciones justificadas....11
E. Integrar las fuentes de datos....13
F. Validar el resultado de la integración....14
G. Presentar el dataset final y los resultados....16
H. Documentar el proceso y la ejecución del código....17
I. Limitaciones y trabajo hacia el proyecto final....18
Conclusiones....19
Referencias....20
Anexos....21

Resumen

El proyecto desarrolla un pipeline de datos reproducible que integra las transacciones de venta de las tiendas Walmart (2022–2025) con un calendario de festivos de seis países, con el fin de analizar la estacionalidad y el efecto de los días festivos sobre las ventas. En este segundo avance se identifican las fuentes, se diagnostica la calidad de los datos antes de modificarlos y se aplican 14 operaciones justificadas técnicamente: 8 de limpieza y 6 de transformación de variables.

Posteriormente, las ventas se integran con el calendario de festivos mediante una unión izquierda por país y fecha, y el resultado se verifica con 11 validaciones automáticas, incluida una verificación cruzada independiente. El dataset consolidado conserva 17,992 transacciones y 46 columnas, mantiene los totales monetarios (ventas por 8,229,473.04 y utilidad por 1,643,056.73) y está listo para el análisis del proyecto final. El código se ejecuta con un solo comando y genera salidas verificables.

Palabras clave: pipeline de datos, calidad de datos, integración, festivos, estacionalidad, ventas.

Introducción

Los registros históricos de ventas, por sí solos, no permiten comprender del todo las variaciones de la demanda. Esas variaciones pueden estar asociadas con días festivos, eventos especiales y otros factores temporales. Para el área de análisis y planificación de ventas, identificar los períodos de alta y baja demanda es un insumo clave para planear inventarios, promociones y estrategias comerciales.

En este caso de estudio se integran dos fuentes de datos heterogéneas: un archivo plano con transacciones de venta y un calendario de festivos generado de forma programática. Antes de unirlas fue necesario evaluar la calidad de los datos, corregir inconsistencias (duplicados, categorías mal escritas, valores imposibles y nulos) y construir variables que faciliten el análisis, como las de calendario, el margen de ganancia y los rangos de edad y de descuento.

El desarrollo de este segundo avance aplica conceptos de extracción, perfilado y limpieza de datos, transformación de variables, integración de tablas por llaves y validación de resultados. Cada decisión técnica se justifica y se respalda con evidencia generada por el propio código, que puede ejecutarse en vivo durante la sustentación.

Objetivo

Diseñar e implementar un pipeline de datos reproducible que extraiga, diagnostique, limpie, transforme e integre las ventas de Walmart con el calendario de festivos de cada país, validando rigurosamente el dataset consolidado para identificar patrones de estacionalidad y el efecto de los días festivos sobre las ventas.

Objetivos específicos de este avance:

1. Identificar con claridad el origen de todas las fuentes de datos utilizadas.
2. Diagnosticar la calidad de los datos antes de modificarlos.
3. Aplicar al menos cinco transformaciones justificadas técnicamente.
4. Integrar las fuentes de forma coherente mediante una llave y una cardinalidad controladas.
5. Validar el dataset consolidado con comprobaciones automáticas.
6. Documentar el proceso y entregar un código ejecutable que genere una salida verificable.

Metodología

El proceso se organizó como un pipeline modular de seis etapas, implementado en Python con las librerías `pandas` (McKinney, 2010) y `holidays`. Cada etapa vive en su propio módulo y cada transformación en una función independiente con una sola responsabilidad, de modo que pueda probarse y explicarse por separado. Las etapas son: extracción, diagnóstico de calidad, limpieza, transformación de variables, integración con festivos y validación de la integración.

Para el tratamiento de los datos se siguieron los principios de datos ordenados propuestos por Wickham (2014) y, para la unión de la tabla de ventas con la de festivos, el enfoque de tabla de hechos y tabla de dimensión descrito por Kimball y Ross (2013). Se adoptó el principio de falla rápida (*fail fast*): si alguna validación no se cumple, el pipeline se detiene y no guarda el resultado.

Contexto

Walmart registra las ventas de sus tiendas, pero los datos históricos no explican por sí solos las variaciones en la demanda. Al integrar esos registros con una fuente externa de festivos es posible identificar patrones de estacionalidad y analizar cómo estos períodos se relacionan con aumentos o disminuciones en las ventas. El usuario interesado es el área de análisis y planificación de ventas.

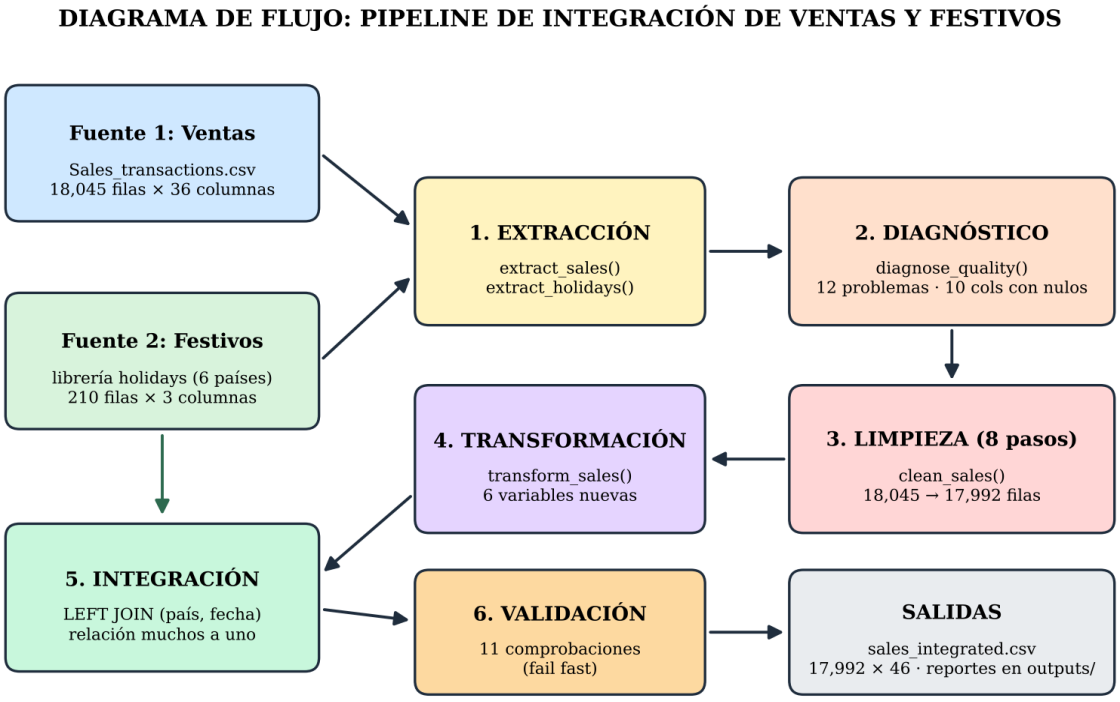
La **pregunta principal** del proyecto es: ¿cómo influyen los días festivos y eventos especiales en el comportamiento de las ventas de las tiendas Walmart? Como preguntas secundarias se plantean qué eventos se asocian con los mayores niveles de ventas, si las ventas aumentan en las semanas con eventos especiales, qué tiendas presentan mayores variaciones, si existen patrones estacionales recurrentes y qué diferencia hay entre semanas normales y semanas con eventos.

Decisión de alcance respecto al Avance 1. En el primer avance la fuente de festivos cubría únicamente Estados Unidos (federal y estatal) y quedó pendiente resolver que las ventas incluyen seis países. En este avance el pendiente se resolvió ampliando la cobertura: el calendario se genera para los seis países presentes en las ventas (Estados Unidos, Reino Unido, Canadá, Alemania, Francia y Australia), de modo que cada venta se cruza con el calendario de su propio país.

Actividades a desarrollar

A. Diagrama de flujo del pipeline de datos

La Gráfica 1 resume el flujo completo del pipeline, desde las dos fuentes hasta las salidas verificables.



Gráfica 1. *Diagrama de Flujo – Pipeline de Integración de Ventas y Festivos*

B. Identificar las fuentes de datos

El proyecto utiliza dos fuentes con origen y formato diferentes, resumidas en la Tabla 1.

#	Fuente	Origen	Formato	Periodo	Volumen
1	`Sales_transactions.csv`	Archivo plano con transacciones a nivel de línea de pedido (cliente, producto, tienda, canal, pago, envío, devoluciones)	CSV	2022–2025	18,045 filas × 36 columnas
2	`holidays_raw.csv`	Generada de forma programática con la librería `holidays` para los 6 países de las ventas	CSV	2022–2025	210 filas × 3 columnas

Tabla 1. *Fuentes de Datos del Proyecto*

Fuente 1 (ventas). Se lee directamente con `pd.read_csv` sin modificarla. Cubre 6 países y 12 tiendas. Por país, el archivo contiene 7,544 filas de Estados Unidos, 2,981 del Reino Unido, 2,685 de Canadá, 2,132 de Alemania, 1,586 de Francia y 1,117 de Australia. El diccionario de datos completo se presentó en el Avance 1.

Fuente 2 (festivos). La función `extract_holidays` recorre los seis países, consulta `holidays.country_holidays(código, years=2022-2025)` y guarda el resultado en `data/raw/holidays_raw.csv` con las columnas `Country`, `Holiday_Date` y `Holiday_Name`. Los festivos marcados como "(observed)" corresponden a festivos trasladados por caer en fin de semana. Los registros por país se presentan en la Tabla 2.

País	Festivos (2022–2025)
Estados Unidos	48
Francia	44
Alemania	36
Reino Unido	30
Australia	29
Canadá	23
Total	**210**

Tabla 2. *Festivos Generados por País*

Llave de integración. Las fuentes se relacionan por **(país, fecha)**: `(Country, Order_Date)` en ventas y `(Country, Holiday_Date)` en festivos. La llave incluye el país porque un mismo día puede ser festivo en un país y no serlo en otro.

C. Diagnosticar la calidad de los datos

El diagnóstico se ejecuta antes de limpiar, mediante la función `diagnose_quality`, que no modifica los datos y guarda dos reportes en la carpeta `outputs/`: `diagnostico_columnas.csv` (tipo, nulos y valores únicos por columna) y

`diagnostico_problemas.csv`. Los problemas concretos encontrados se muestran en la Tabla 3.

Problema	Cantidad
Filas duplicadas exactas	45
`Transaction_ID` repetido	45
`Quantity` ≤ 0	8
`Customer_Age` fuera del rango 18–90	4
`Discount_Percentage` > 100	1
`Profit` $\neq$ `Sales_Amount` − `Cost_Amount`	110
Variantes de escritura en `Customer_Gender`	3
Variantes de escritura en `Order_Status`	3
Variantes de escritura en `Product_Category`	3
Variantes de escritura en `Payment_Method`	4
Variantes de escritura en `City`	3
Variantes de escritura en `Region`	2

Tabla 3. *Problemas de Calidad Detectados en la Fuente de Ventas*

Además, 10 columnas presentan valores nulos (Tabla 4). Los nulos de `Return_Reason` y `Promotion_Code` son estructurales, es decir, esperados cuando no hubo devolución o no se aplicó una promoción; los demás son datos faltantes que deben tratarse.

Columna	Nulos	%	Naturaleza
`Return_Reason`	16,168	89.60	Estructural (sin devolución)
`Promotion_Code`	9,737	53.96	Estructural (sin promoción)
`Customer_Rating`	3,435	19.04	Dato faltante real
`Customer_Age`	218	1.21	Dato faltante
`Inventory_Level`	198	1.10	Dato faltante
`Delivery_Days`	162	0.90	Dato faltante
`Customer_Gender`	162	0.90	Dato faltante
`Payment_Method`	147	0.81	Dato faltante
`Product_Name`	144	0.80	Dato faltante
`Customer_Name`	108	0.60	Dato faltante

Tabla 4. *Valores Nulos por Columna*

El diagnóstico confirma los hallazgos del Avance 1 (45 duplicados, inconsistencias de formato, outliers y nulos). Al automatizarlo se precisaron los umbrales: la edad se valida en el rango 18–90 (4 registros) y se revisan todas las cantidades menores o iguales a cero (8 registros). El Avance 1 también reportó 77 registros cuyo `Sales_Amount` difiere de `Quantity × Unit_Price × (1 − Descuento/100)`; esa regla permanece pendiente (véase la actividad I).

D. Aplicar las transformaciones justificadas

El pipeline aplica dos grupos de operaciones sobre las ventas: la limpieza (`clean_sales`), que deja un registro en `outputs/log_limpieza.txt`, y la transformación de variables (`transform_sales`). En total son 14 operaciones, cada una con su justificación técnica, lo que supera el mínimo de cinco exigido.

Limpieza (8 operaciones)

#	Operación	Resultado	Justificación técnica
L1	Eliminar duplicados exactos	45 filas eliminadas	Una transacción repetida infla ventas y ganancias. Al ser filas idénticas, conservar una no pierde información.
L2	Quitar espacios al inicio y final de las columnas de texto	Todas las columnas de texto	Evita que `"PayPal "` y `"PayPal"` se cuenten como categorías distintas.
L3	Estandarizar categorías (pago, género, estado, categoría, ciudad y región)	`credit card`, `DebitCard`, `paypal`, etc. quedan en un único valor	Las variantes de escritura duplicarían segmentos en cualquier agrupación.
L4	Pasar valores imposibles a nulo	4 edades fuera de 18–90 y 1 descuento > 100 %	Son errores de captura que no pueden corregirse con certeza; se tratan como dato faltante.
L5	Eliminar filas con `Quantity` ≤ 0	8 filas eliminadas	Una cantidad cero o negativa no es una venta válida y no se puede reconstruir la cantidad real.
L6	Recuperar nulos desde otra columna	`Customer_Name` por `Customer_ID`; `Product_Name` por `Product_ID`	El ID identifica de forma única al cliente o producto, por lo que el dato se recupera sin inventarlo.
L7	Imputar nulos	Mediana (edad, descuento); mediana por método de envío (`Delivery_Days`); mediana por producto (`Inventory_Level`); etiquetas `Unknown`, `No Promo`, `No Return`	La mediana resiste valores extremos; los días de entrega dependen del envío y el inventario del producto. `Customer_Rating` se deja sin imputar para no sesgar el análisis con opiniones inventadas.
L8	Recalcular `Profit` = `Sales_Amount` − `Cost_Amount`	110 filas no coincidían	La utilidad está definida como esa diferencia; recalcularla es determinístico y garantiza coherencia.

Tabla 5. *Operaciones de Limpieza y su Justificación*

El efecto neto de la limpieza es una reducción de 18,045 a 17,992 filas (45 duplicados y 8 cantidades inválidas). La Tabla 6 compara los nulos antes y después.

Columna	Antes	Después
`Customer_Name`, `Product_Name`, `Customer_Gender`, `Payment_Method`	108 / 144 / 162 / 147	0
`Customer_Age`, `Delivery_Days`, `Inventory_Level`	218 / 162 / 198	0
`Return_Reason`, `Promotion_Code`	16,168 / 9,737	0 (etiquetados)
`Customer_Rating`	3,435	3,430 (decisión deliberada; 5 filas salieron con la limpieza)

Tabla 6. *Valores Nulos Antes y Después de la Limpieza*

Transformación de variables (6 operaciones)

#	Transformación	Columnas nuevas	Justificación técnica
T1	Convertir `Order_Date` de texto a fecha	`Order_Date` tipo `datetime`	Con texto no se puede extraer mes ni día de la semana, y para unir con los festivos ambas fechas deben ser del mismo tipo.
T2	Variables de calendario	`Order_Month`, `Order_Quarter`, `Order_Weekday`, `Is_Weekend`	Permiten analizar estacionalidad por mes, trimestre y fin de semana, algo imposible con una sola fecha.
T3	Agrupar la edad en rangos	`Age_Group` (18-25, 26-35, 36-45, 46-55, 56+)	Pasar de decenas de edades a cinco grupos permite comparar segmentos de clientes con claridad.
T4	Clasificar el descuento	`Discount_Level` (No Discount, Low 1-10 %, High > 10 %)	Gran parte de las ventas no tiene descuento y el resto se concentra en pocos valores; una categoría es más fácil de comparar.
T5	Calcular el margen porcentual	`Profit_Margin_Pct`	La ganancia en dinero no permite comparar un producto barato con uno caro; el margen sí.
T6	Codificar la devolución como 1/0	`Is_Returned`	Con 1/0 se puede sumar y promediar directamente; el promedio es la tasa de devolución.

Tabla 7. *Transformaciones de Variables y su Justificación*

La Tabla 8 muestra la distribución resultante de las variables creadas en T3 y T4. A modo de ejemplo, la transformación T5 se implementa como una función independiente:

Age_Group	Filas	Discount_Level	Filas
36-45	6,487	No Discount	7,634
26-35	4,797	Low (1-10%)	6,784
46-55	3,591	High (>10%)	3,574
18-25	1,960		
56+	1,157		

Tabla 8. *Distribución de `Age_Group` y `Discount_Level`*

```
def add_profit_margin(df: pd.DataFrame) -> pd.DataFrame:
    """T5. Calcular el margen de ganancia en porcentaje."""
    df = df.copy()
    df["Profit_Margin_Pct"] = (df["Profit"] / df["Sales_Amount"] * 100).round(2)
    return df
```

E. Integrar las fuentes de datos

Las ventas transformadas (17,992 filas) se integran con la dimensión de festivos mediante una unión izquierda (`LEFT JOIN`), de modo que se conservan todas las ventas, tengan o no festivo. Las decisiones de diseño se resumen en la Tabla 9.

Elemento	Decisión
Llave	`(Country, Order_Date)` = `(Country, Holiday_Date)`
Tipo de unión	`LEFT JOIN`, con las ventas a la izquierda
Cardinalidad	Muchos a uno: muchas ventas por día y país, un festivo por día y país
Control	`validate="many_to_one"`: pandas lanza un error si la cardinalidad no se cumple
Columnas nuevas	`Is_Holiday` (booleana) y `Holiday_Name` ("No Holiday" si no aplica)

Tabla 9. *Diseño de la Integración*

Antes de unir, la función `prepare_holidays` convierte la fecha al mismo tipo que `Order_Date`, limpia el país y consolida en una sola fila los festivos que coinciden el mismo día y país (nombres separados por " | "). Sin este paso, un día con dos festivos duplicaría las ventas de ese día.

```
merged = df.merge(
    holidays_dim,
    how="left",
    left_on=["Country", "Order_Date"],
    right_on=["Country", "Holiday_Date"],
    validate="many_to_one",
    indicator=True,
)
merged["Is_Holiday"] = merged["_merge"].eq("both")
merged["Holiday_Name"] = merged["Holiday_Name"].fillna("No Holiday")
```

Como resultado, 498 de las 17,992 ventas (2.77 %) ocurrieron en un día festivo de su país (Tabla 10).

País	Ventas en festivo	Total de ventas	% en festivo
Estados Unidos	237	7,526	3.15
Reino Unido	74	2,972	2.49
Alemania	69	2,125	3.25
Francia	56	1,578	3.55
Canadá	41	2,678	1.53
Australia	21	1,113	1.89
Total	**498**	**17,992**	**2.77**

Tabla 10. *Ventas en Día Festivo por País*

Granularidad. En el Avance 1 se planteó homologar la fecha a nivel de semana ISO. En este avance se prefirió la unión por fecha exacta, porque es más precisa y no pierde información. La semana ISO puede derivarse después sobre el dataset integrado para el análisis de "semanas con evento" del proyecto final.

F. Validar el resultado de la integración

La función `validate_integration` ejecuta 11 comprobaciones automáticas, antes y después de la unión, y guarda el reporte en `outputs/validacion_integracion.csv`. Si alguna falla, el pipeline se detiene con un error, por lo que un dataset inconsistente nunca llega a guardarse. Los resultados se presentan en la Tabla 11.

#	Validación	Momento	Resultado	Detalle
V1	`Transaction_ID` único en ventas	Antes	OK	0 duplicados
V2	Llave de festivos única	Antes	OK	0 duplicados
V3	Todo país de ventas tiene calendario	Antes	OK	Sin países faltantes
V4	Todo año de ventas está cubierto	Antes	OK	Sin años faltantes
V5	El join no agrega ni quita filas	Después	OK	17,992 antes y después
V6	Sin multiplicación de filas	Después	OK	0 transacciones repetidas
V7	Mismo conjunto de `Transaction_ID`	Después	OK	No se pierde ni se inventa ninguna
V8	Totales de ventas y utilidad conservados	Después	OK	Ventas 8,229,473.04; utilidad 1,643,056.73
V9	`Is_Holiday` booleana y coherente con `Holiday_Name`	Después	OK	Sin nulos
V10	Verificación cruzada del flag (sin merge)	Después	OK	Método independiente
V11	Columnas originales intactas	Después	OK	—

Tabla 11. *Validaciones de la Integración*

Resultado: 11 de 11 validaciones aprobadas. La verificación cruzada (V10) es la más importante: recalcula `Is_Holiday` con un conjunto de pares (país, fecha) sin usar `merge`, de modo que el indicador se comprueba con un método distinto del que lo generó.

La Tabla 12 muestra el estado de las reglas de calidad del Avance 1 después de la limpieza.

Regla	Estado tras la limpieza
R1. Unicidad de la transacción (45 incumplían)	Resuelta: 0 duplicados (V1)
R2. Coherencia aritmética de `Sales_Amount` (77 incumplen)	Pendiente (véase la actividad I)
R3. Valores válidos de cantidad y descuento (9 incumplían)	Resuelta: 8 filas eliminadas y 1 descuento imputado
R4. Fechas coherentes	Se mantiene: del 2022-01-01 al 2025-12-31, sin fechas fuera de periodo

Tabla 12. *Estado de las Reglas de Calidad del Avance 1*

G. Presentar el dataset final y los resultados

El dataset consolidado se guarda en `data/processed/sales_integrated.csv`. Sus características se resumen en la Tabla 13; junto con él se generan `sales_clean.csv` y `sales_transformed.csv`.

Característica	Valor
Dimensiones	17,992 filas × 46 columnas
Composición de columnas	36 originales + 8 transformadas + 2 de integración
Periodo	2022-01-01 a 2025-12-31
Ventas totales	8,229,473.04
Utilidad total	1,643,056.73
Nulos restantes	Solo `Customer_Rating` (3,430)

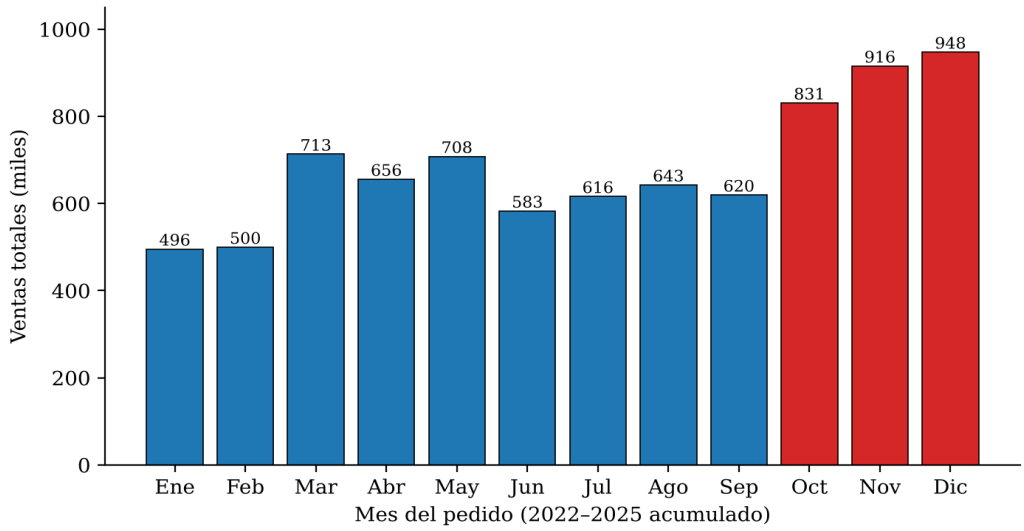
Tabla 13. *Características del Dataset Final*

Los hallazgos preliminares son descriptivos y sirven para comprobar que el dataset integrado permite responder la pregunta del proyecto; el análisis formal queda para la entrega final (Tabla 14).

Métrica	Día sin festivo	Día festivo
Transacciones	17,494	498
Venta promedio por transacción	456.84	477.09
Cantidad promedio por transacción	2.13	2.19
Margen promedio (%)	24.35	22.94

Tabla 14. *Comparación entre Días Festivos y No Festivos*

La venta promedio por transacción en días festivos es aproximadamente 4.4 % mayor, mientras que el margen promedio es 1.4 puntos porcentuales menor, un comportamiento compatible con más descuentos o promociones en esas fechas, hipótesis que se evaluará con `Discount_Level`. Además, la Gráfica 2 evidencia una estacionalidad mensual clara: octubre, noviembre y diciembre concentran las mayores ventas, y enero y febrero las menores.



Gráfica 2. *Ventas Totales por Mes del Pedido*

Entre los festivos con mayores ventas acumuladas se encuentran las variantes de Navidad por país (Christmas Day, Erster Weihnachtstag y Boxing Day), Año Nuevo y Thanksgiving.

Cautela: solo el 2.77 % de las transacciones cae en un día festivo y varios festivos tienen pocas observaciones (entre 9 y 49), por lo que estas diferencias no deben considerarse concluyentes sin una prueba estadística.

H. Documentar el proceso y la ejecución del código

El proyecto se organiza en módulos por etapa. La estructura de carpetas es la siguiente:

```
VentasWalmart-integracion-csv/
├── main.py                  # Orquesta todo el pipeline
├── requirements.txt         # pandas, holidays
├── data/
│   ├── raw/                 # Sales_transactions.csv, holidays_raw.csv
│   └── processed/           # sales_clean, sales_transformed, sales_integrated
├── outputs/                 # Reportes de diagnóstico, log y validaciones
├── notebooks/               # exploración de festivos (EE. UU.)
└── src/
    ├── extraction.py        # Extracción de ventas y festivos
    ├── cleaning.py          # Diagnóstico de calidad y limpieza
    ├── transformation.py    # Transformaciones de variables
    ├── integration.py       # Unión ventas + festivos
    └── validation.py        # Validaciones de la integración
```

Para ejecutar el pipeline completo se usan los siguientes comandos:

```
pip install -r requirements.txt
python main.py
```

La ejecución genera `outputs/diagnostico_columnas.csv`, `outputs/diagnostico_problemas.csv`, `outputs/log_limpieza.txt`, `outputs/validacion_integracion.csv` y los archivos de `data/processed/`. Se verificó que la ejecución en una copia limpia del proyecto termina sin errores y reproduce las cifras de este informe. El código completo se anexa al final del documento.

Las decisiones de diseño principales fueron: separar cada etapa en un módulo y cada transformación en una función; no imputar `Customer_Rating` para no sesgar el análisis; eliminar solo lo irrecuperable (duplicados exactos y cantidades no positivas) y corregir o etiquetar el resto; detener el pipeline si una validación falla; y guardar como archivos el diagnóstico, el log de limpieza y el reporte de validaciones para garantizar la trazabilidad.

I. Limitaciones y trabajo hacia el proyecto final

- **Coherencia aritmética (R2).** Los 77 registros donde `Sales_Amount` difiere del valor esperado se detectaron en el Avance 1 y todavía no se corrigen en el pipeline. Es una corrección determinística que se incorporará como una nueva función de limpieza.
- **Análisis semanal.** Derivar la semana ISO y un indicador de “semana con festivo” para responder las preguntas sobre semanas normales frente a semanas con eventos.
- **Festivos regionales.** La fuente actual contiene festivos nacionales; los festivos estatales de EE. UU. generados en el Avance 1 podrían integrarse usando `Region`.
- **Reproducibilidad.** Fijar las versiones de `pandas` y `holidays` en `requirements.txt`, completar el README y agregar pruebas automáticas.

- **Cronograma.** Corregir observaciones (9–10 de noviembre), mejorar el pipeline (11–12), pruebas cruzadas en entorno limpio (13–14), README e informe técnico (15–16) y sustentación final (17–18).

Conclusiones

El pipeline extrae las dos fuentes, diagnostica y corrige los problemas de calidad, aplica 14 operaciones justificadas, integra las ventas con el calendario de festivos de cada país y valida el resultado con 11 comprobaciones automáticas. El dataset consolidado conserva todas las transacciones válidas y los totales monetarios, y queda listo para el análisis de estacionalidad y del efecto de los festivos en el proyecto final.

El diagnóstico previo permitió cuantificar los problemas (duplicados, categorías inconsistentes, valores imposibles y nulos) y tratarlos con reglas explícitas, diferenciando lo que puede corregirse de forma determinística de lo que debe eliminarse o dejarse sin imputar. La limpieza redujo el dataset de 18,045 a 17,992 filas sin comprometer la información útil.

La integración por país y fecha, con cardinalidad muchos a uno controlada, evitó la multiplicación de filas y fue comprobada con una verificación cruzada independiente. Los resultados preliminares sugieren una estacionalidad marcada en el último trimestre del año y una venta promedio ligeramente mayor en días festivos, aunque estos hallazgos requieren una validación estadística.

Queda pendiente corregir las 77 inconsistencias aritméticas de `Sales_Amount`, derivar la semana ISO para el análisis de semanas con evento y completar la documentación para la entrega final.

Referencias

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.

McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Python Holidays contributors. (s. f.). *holidays* [Software]. Python Package Index. https://pypi.org/project/holidays/

Wickham, H. (2014). Tidy data. *Journal of Statistical Software, 59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10

Anexos

A continuación se incluye el código fuente completo del pipeline, que se ejecuta con `python main.py`.

Anexo A. main.py

```
from pathlib import Path

from src.extraction import extract sales, extract holidays
from src.cleaning import diagnose quality, clean sales
from src.transformation import transform_sales
from src.integration import integrate_sales_holidays
from src.validation import validate integration

BASE DIR = Path(  file  ).resolve().parent
PROCESSED DIR = BASE DIR / "data" / "processed"
OUTPUTS_DIR = BASE_DIR / "outputs"



def main():
    # PASO 1: EXTRACCIÓN
    sales = extract_sales()
    holidays_df = extract_holidays()

    # PASO 2: DIAGNÓSTICO Y LIMPIEZA
    diagnose_quality(sales)
    sales_clean = clean_sales(sales)

    # PASO 3: TRANSFORMACIÓN
    sales_transformed = transform_sales(sales_clean)

    # PASO 4: INTEGRACIÓN CON FESTIVOS
    sales_integrated = integrate_sales_holidays(sales_transformed, holidays_df)

    # PASO 5: VALIDACIÓN DE LA INTEGRACIÓN
    report = validate_integration(sales_transformed, holidays_df, sales_integrated)

    # PASO 6: GUARDAR LOS RESULTADOS
    PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
    OUTPUTS_DIR.mkdir(parents=True, exist_ok=True)

    sales transformed.to csv(PROCESSED DIR / "sales transformed.csv", index=False)
    print(f"\nGuardado: data/processed/sales_transformed.csv ({len(sales_transformed)} filas)")

    sales integrated.to csv(PROCESSED DIR / "sales integrated.csv", index=False)
    print(f"Guardado: data/processed/sales_integrated.csv ({len(sales_integrated)} filas)")

    report.to csv(OUTPUTS DIR / "validacion integracion.csv", index=False)
    print("Guardado: outputs/validacion_integracion.csv")



if __name__ == "__main__":
    main()
```

Anexo B. src/extraction.py

```
"""
extraction.py
Extrae las dos fuentes de datos del proyecto:
  1. CSV de ventas (archivo plano)  -> data/raw/Sales_transactions.csv
```

```
  2. Festivos generados con la librería `holidays` -> data/raw/holidays_raw.csv
"""


from pathlib import Path


import holidays
import pandas as pd


# Carpeta raíz del proyecto (dos niveles arriba de este archivo)
BASE DIR = Path(  file  ).resolve().parent.parent
RAW_DIR = BASE_DIR / "data" / "raw"


SALES FILE = RAW DIR / "Sales transactions.csv"
HOLIDAYS_FILE = RAW_DIR / "holidays_raw.csv"


# Países del CSV -> código que usa la librería holidays
COUNTRY_CODES = {
    "United States": "US",
    "United Kingdom": "GB",
    "Canada": "CA",
    "Germany": "DE",
    "France": "FR",
    "Australia": "AU",
}


YEARS = range(2022, 2026)  # 2022 a 2025



def extract_sales() -> pd.DataFrame:
    """Lee el CSV de ventas tal como viene, sin modificarlo."""
    if not SALES FILE.exists():
        raise FileNotFoundError(f"No se encontró el archivo: {SALES FILE}")
    df = pd.read_csv(SALES_FILE)
    print(f"[extraction] Ventas: {df.shape[0]} filas, {df.shape[1]} columnas")
    return df



def extract_holidays() -> pd.DataFrame:
    """Genera los festivos de cada país con la librería `holidays`
    y los guarda en data/raw/holidays raw.csv."""
    records = []
    for country, code in COUNTRY_CODES.items():
        country_holidays = holidays.country_holidays(code, years=YEARS)
        for date, name in country_holidays.items():
            records.append(
                {"Country": country, "Holiday_Date": date, "Holiday_Name": name}
            )


    df = pd.DataFrame(records).sort_values(["Country", "Holiday_Date"])
    df.to_csv(HOLIDAYS_FILE, index=False)
    print(f"[extraction] Festivos: {df.shape[0]} filas guardadas en {HOLIDAYS FILE.name}")
    return df
```

Anexo C. src/cleaning.py

```
"""
cleaning.py
1) Diagnóstico de calidad (ANTES de limpiar) -> outputs/
2) Limpieza del dataset de ventas
"""


from pathlib import Path


import pandas as pd


BASE_DIR = Path(__file__).resolve().parent.parent
OUTPUT_DIR = BASE_DIR / "outputs"
```

```
# Columnas de texto categórico donde buscamos variantes mal escritas
CATEGORICAL_COLS = [
    "Customer Gender", "Order Status", "Product Category",
    "Payment Method", "City", "Region",
]



# ---------------------------------------------------------------------
# 1. DIAGNÓSTICO
# ---------------------------------------------------------------------
def diagnose_quality(df: pd.DataFrame) -> pd.DataFrame:
    """Revisa la calidad del dataset SIN modificarlo y guarda dos reportes."""
    OUTPUT_DIR.mkdir(exist_ok=True)

    # (a) Reporte por columna: tipo, nulos y valores únicos
    columns_report = pd.DataFrame({
        "Tipo": df.dtypes.astype(str),
        "Nulos": df.isna().sum(),
        "Pct_Nulos": (df.isna().mean() * 100).round(2),
        "Valores_Unicos": df.nunique(),
    })
    columns_report.index.name = "Columna"
    columns_report.to_csv(OUTPUT_DIR / "diagnostico_columnas.csv")

    # (b) Reporte de problemas concretos
    issues = [
        ("Filas duplicadas exactas", int(df.duplicated().sum())),
        ("Transaction ID repetido", int(df["Transaction ID"].duplicated().sum())),
        ("Quantity <= 0", int((df["Quantity"] <= 0).sum())),
        ("Customer_Age fuera de 18-90", int(((df["Customer_Age"] < 18) | (df["Customer_Age"] >
90)).sum())),
        ("Discount Percentage > 100", int((df["Discount Percentage"] > 100).sum())),
        ("Profit != Sales - Cost", int(((df["Sales Amount"] - df["Cost Amount"] -
df["Profit"]).abs() > 0.02).sum())),
    ]
    # Variantes de escritura: valores distintos vs. valores distintos "normalizados"
    for col in CATEGORICAL_COLS:
        raw = df[col].dropna().nunique()
        norm = df[col].dropna().str.strip().str.lower().str.replace(r"[\s\-]", "",
regex=True).nunique()
        issues.append((f"Variantes de escritura en {col}", raw - norm))

    issues_report = pd.DataFrame(issues, columns=["Problema", "Cantidad"])
    issues_report.to_csv(OUTPUT_DIR / "diagnostico_problemas.csv", index=False)

    print("\n[cleaning] ===== DIAGNÓSTICO DE CALIDAD =====")
    print(issues_report.to_string(index=False))
    print("\nColumnas con nulos:")
    print(columns_report[columns_report["Nulos"] > 0][["Nulos", "Pct_Nulos"]])
    return issues_report



# ---------------------------------------------------------------------
# 2. LIMPIEZA
# ---------------------------------------------------------------------
def clean_sales(df: pd.DataFrame) -> pd.DataFrame:
    """Devuelve una copia limpia del dataset. Cada paso queda en un log."""
    df = df.copy()
    log = []
    rows_start = len(df)

    # --- 1. Duplicados ------------------------------------------------
    # Una misma transacción repetida infla ventas y ganancias.
    n = int(df.duplicated().sum())
    df = df.drop_duplicates().reset_index(drop=True)
    log.append(f"Duplicados exactos eliminados: {n}")

    # --- 2. Espacios sobrantes en texto -------------------------------
```

```
    text_cols = df.select_dtypes(include=["object", "string"]).columns
    for col in text_cols:
        df[col] = df[col].str.strip()
    log.append("Espacios al inicio/final eliminados en columnas de texto")


    # --- 3. Estandarizar categorías ------------------------------------
    # 'paypal' y 'PayPal' son lo mismo, pero Python las cuenta como distintas.
    payment_map = {
        "creditcard": "Credit Card", "debitcard": "Debit Card",
        "paypal": "PayPal", "applepay": "Apple Pay",
        "cash": "Cash", "banktransfer": "Bank Transfer",
    }
    key = df["Payment_Method"].str.lower().str.replace(r"\s", "", regex=True)
    df["Payment_Method"] = key.map(payment_map)


    gender_map = {"male": "Male", "female": "Female",
                  "nonbinary": "Non-binary"}
    key = df["Customer_Gender"].str.lower().str.replace(r"[\s\-]", "", regex=True)
    df["Customer_Gender"] = key.map(gender_map)


    df["Order_Status"] = df["Order_Status"].str.capitalize()


    category_map = {"electronics": "Electronics", "furniture": "Furniture",
                    "appliances": "Appliances", "office supplies": "Office Supplies"}
    df["Product_Category"] = df["Product_Category"].str.lower().map(category_map)


    df["City"] = df["City"].str.title()
    df["Region"] = df["Region"].replace({
        "NRW": "North Rhine-Westphalia",
        "British columbia": "British Columbia",
        "North East": "Northeast",   # en EE.UU. (no en Reino Unido)
    })
    log.append("Categorías estandarizadas: pago, género, estado, categoría, ciudad y región")


    # --- 4. Valores imposibles -> se tratan como nulos ----------------
    # Edades <18 o >90 y descuentos >100% son errores de captura.
    bad_age = (df["Customer_Age"] < 18) | (df["Customer_Age"] > 90)
    df.loc[bad_age, "Customer_Age"] = pd.NA
    bad_disc = df["Discount_Percentage"] > 100
    df.loc[bad disc, "Discount Percentage"] = pd.NA
    log.append(f"Edades inválidas pasadas a nulo: {int(bad age.sum())}; descuentos >100%:
{int(bad_disc.sum())}")


    # Cantidad <= 0 no es una venta válida: se eliminan esas filas.
    bad_qty = df["Quantity"] <= 0
    log.append(f"Filas con Quantity <= 0 eliminadas: {int(bad_qty.sum())}")
    df = df[~bad_qty].reset_index(drop=True)


    # --- 5. Valores nulos ----------------------------------------------
    # 5a. Recuperar el dato desde otra columna que lo identifica de forma única.
    name_by_customer = df.groupby("Customer_ID")["Customer_Name"].first()
    df["Customer_Name"] = df["Customer_Name"].fillna(df["Customer_ID"].map(name_by_customer))


    name_by_product = df.groupby("Product_ID")["Product_Name"].first()
    df["Product Name"] = df["Product Name"].fillna(df["Product ID"].map(name by product))


    # 5b. Imputar con la mediana (resistente a valores extremos).
    df["Customer Age"] = df["Customer Age"].fillna(df["Customer Age"].median()).round()
    df["Discount Percentage"] =
df["Discount_Percentage"].fillna(df["Discount_Percentage"].median())
    # Los días de entrega dependen del método de envío (Next Day vs Standard).
    df["Delivery Days"] = df["Delivery Days"].fillna(
        df.groupby("Shipping Method")["Delivery Days"].transform("median"))
    df["Inventory_Level"] = df["Inventory_Level"].fillna(
        df.groupby("Product_ID")["Inventory_Level"].transform("median"))


    # 5c. Categorías sin dato -> etiqueta explícita.
    for col in ["Customer_Name", "Customer_Gender", "Payment_Method"]:
        df[col] = df[col].fillna("Unknown")
```

```
    # Nulo esperado: sin promoción / sin devolución no es un error.
    df["Promotion Code"] = df["Promotion Code"].fillna("No Promo")
    df["Return_Reason"] = df["Return_Reason"].fillna("No Return")


    # Customer Rating se deja nulo a propósito: inventar la opinión de un
    # cliente sesgaría el análisis.
    log.append("Nulos tratados: recuperados por ID, mediana, etiqueta 'Unknown'/'No Promo'/'No
Return'. "
               "Customer_Rating se deja sin imputar")


    # --- 6. Coherencia numérica ----------------------------------------
    n = int(((df["Sales_Amount"] - df["Cost_Amount"] - df["Profit"]).abs() > 0.02).sum())
    df["Profit"] = (df["Sales Amount"] - df["Cost Amount"]).round(2)
    log.append(f"Profit recalculado como Sales - Cost ({n} filas no coincidían)")


    log.append(f"Filas: {rows_start} -> {len(df)}")


    # Guardar el log y mostrarlo
    OUTPUT_DIR.mkdir(exist_ok=True)
    (OUTPUT_DIR / "log_limpieza.txt").write_text("\n".join(log), encoding="utf-8")
    print("\n[cleaning] ===== LIMPIEZA =====")
    for line in log:
        print(" -", line)
    return df
```

Anexo D. src/transformation.py

```
"""
transformation.py
Transformaciones sobre el dataset de ventas YA limpio.
Cada función hace UNA transformación y explica por qué.
"""


import pandas as pd



def convert_order_date(df: pd.DataFrame) -> pd.DataFrame:
    """T1. Convertir Order_Date de texto a fecha real.
    Por qué: con texto no se puede sacar mes ni día de la semana, y para
    unir con la tabla de festivos las dos fechas deben ser del mismo tipo."""
    df = df.copy()
    df["Order_Date"] = pd.to_datetime(df["Order_Date"], format="%Y-%m-%d")
    return df



def add date features(df: pd.DataFrame) -> pd.DataFrame:
    """T2. Crear mes, trimestre, día de la semana y fin de semana.
    Por qué: permiten analizar estacionalidad (¿se vende más en diciembre?
    ¿más en fin de semana?), algo imposible con una sola fecha."""
    df = df.copy()
    df["Order Month"] = df["Order Date"].dt.month
    df["Order Quarter"] = df["Order Date"].dt.quarter
    df["Order_Weekday"] = df["Order_Date"].dt.day_name()
    df["Is_Weekend"] = (df["Order_Date"].dt.dayofweek >= 5).astype(int)
    return df



def add_age_group(df: pd.DataFrame) -> pd.DataFrame:
    """T3. Agrupar la edad en rangos.
    Por qué: 60 edades distintas son difíciles de analizar; en 5 rangos
    se comparan grupos de clientes con claridad."""
    df = df.copy()
    df["Age_Group"] = pd.cut(
        df["Customer_Age"],
        bins=[17, 25, 35, 45, 55, 100],
        labels=["18-25", "26-35", "36-45", "46-55", "56+"],
    ).astype(str)
    return df
```

```
def add_discount_level(df: pd.DataFrame) -> pd.DataFrame:
    """T4. Clasificar el descuento en Sin descuento / Bajo / Alto.
    Por qué: el 42 % de las ventas no tiene descuento y el resto se concentra
    en pocos valores; una categoría es más fácil de comparar que un número."""
    df = df.copy()
    df["Discount_Level"] = pd.cut(
        df["Discount_Percentage"],
        bins=[-1, 0, 10, 100],
        labels=["No Discount", "Low (1-10%)", "High (>10%)"],
    ).astype(str)
    return df


def add_profit_margin(df: pd.DataFrame) -> pd.DataFrame:
    """T5. Calcular el margen de ganancia en porcentaje.
    Por qué: la ganancia en dinero no permite comparar un producto de $20
    con uno de $1.400; el margen (Profit / Ventas) sí."""
    df = df.copy()
    df["Profit_Margin_Pct"] = (df["Profit"] / df["Sales_Amount"] * 100).round(2)
    return df


def encode_return_flag(df: pd.DataFrame) -> pd.DataFrame:
    """T6. Convertir Return Flag de Yes/No a 1/0.
    Por qué: con 1/0 se puede sumar y promediar directamente
    (el promedio es la tasa de devolución)."""
    df = df.copy()
    df["Is_Returned"] = (df["Return_Flag"] == "Yes").astype(int)
    return df


def transform_sales(df: pd.DataFrame) -> pd.DataFrame:
    """Aplica todas las transformaciones en orden."""
    steps = [
        convert_order_date,
        add_date_features,
        add_age_group,
        add_discount_level,
        add_profit_margin,
        encode_return_flag,
    ]
    cols_before = set(df.columns)
    for step in steps:
        df = step(df)

    new_cols = sorted(set(df.columns) - cols_before)
    print("\n[transformation] ===== TRANSFORMACIONES =====")
    print(f" - {len(steps)} transformaciones aplicadas")
    print(f" - Columnas nuevas: {new_cols}")
    return df
```

Anexo E. src/integration.py

```
"""
integration.py
Integra las ventas transformadas con la tabla de festivos.
Resultado: cada venta queda marcada con Is_Holiday (True/False)
y el nombre del festivo, si lo hay.

Llave de unión: (Country, fecha)  -> relación muchos-a-uno (m:1)
Tipo de unión:  LEFT JOIN (se conservan todas las ventas)
"""


import pandas as pd
```

```
def prepare_holidays(holidays_df: pd.DataFrame) -> pd.DataFrame:
    """I1. Dejar la tabla de festivos lista para unir.
    Por qué: la fecha debe ser del mismo tipo que Order Date, y la llave
    (Country, Holiday Date) debe ser única. Si un país tuviera dos festivos
    el mismo día, el join duplicaría esas ventas; por eso se consolidan
    los nombres en una sola fila."""
    df = holidays df.copy()
    df["Country"] = df["Country"].str.strip()
    df["Holiday_Date"] = pd.to_datetime(df["Holiday_Date"], format="%Y-%m-%d")
    df = (
        df.groupby(["Country", "Holiday_Date"], as_index=False)
        .agg(Holiday_Name=("Holiday_Name", lambda x: " | ".join(sorted(set(x)))))
    )
    return df



def add_holiday_flag(sales: pd.DataFrame, holidays_dim: pd.DataFrame) -> pd.DataFrame:
    """I2. Unir ventas con festivos y crear Is_Holiday.
    Por qué: se une por país Y fecha, porque un mismo día puede ser festivo
    en un país y no en otro. validate='many_to_one' hace que pandas lance
    un error si la relación no es m:1 (evita la multiplicación de filas)."""
    df = sales.copy()
    df["Country"] = df["Country"].str.strip()
    df["Order_Date"] = pd.to_datetime(df["Order_Date"])


    merged = df.merge(
        holidays_dim,
        how="left",
        left on=["Country", "Order Date"],
        right on=["Country", "Holiday Date"],
        validate="many_to_one",
        indicator=True,
    )
    merged["Is Holiday"] = merged[" merge"].eq("both")
    merged["Holiday_Name"] = merged["Holiday_Name"].fillna("No Holiday")
    return merged.drop(columns=["Holiday_Date", "_merge"])



def integrate_sales_holidays(sales: pd.DataFrame, holidays_df: pd.DataFrame) -> pd.DataFrame:
    """Aplica la integración completa."""
    holidays_dim = prepare_holidays(holidays_df)
    integrated = add_holiday_flag(sales, holidays_dim)


    n_fest = int(integrated["Is_Holiday"].sum())
    print("\n[integration] ===== INTEGRACIÓN VENTAS + FESTIVOS =====")
    print(f" - Festivos únicos (país, fecha): {len(holidays_dim)}")
    print(f" - Ventas en festivo: {n_fest} de {len(integrated)} ({n_fest /
len(integrated):.2%})")
    print(" - Columnas nuevas: ['Holiday_Name', 'Is_Holiday']")
    return integrated
```

Anexo F. src/validation.py

```
"""
validation.py
Validaciones de la integración ventas + festivos.
Cada función revisa UNA regla y devuelve (nombre, ok, detalle).
validate integration() las ejecuta todas y lanza un error si alguna falla.
"""


import pandas as pd


from src.integration import prepare_holidays



# ------------------------------------------------------------------------
# Validaciones PREVIAS al join
# ------------------------------------------------------------------------
def check_sales_key_unique(sales: pd.DataFrame):
```

```
    """V1. Transaction_ID único en ventas (es el grano de la tabla)."""
    dups = int(sales["Transaction ID"].duplicated().sum())
    return "Transaction_ID único en ventas", dups == 0, f"{dups} duplicados"



def check holiday key unique(holidays dim: pd.DataFrame):
    """V2. Llave (Country, Holiday Date) única: si no, el join multiplica filas."""
    dups = int(holidays dim.duplicated(["Country", "Holiday Date"]).sum())
    return "Llave de festivos única", dups == 0, f"{dups} duplicados"



def check_country_coverage(sales: pd.DataFrame, holidays_dim: pd.DataFrame):
    """V3. Todo país de ventas tiene calendario de festivos."""
    missing = set(sales["Country"].str.strip()) - set(holidays_dim["Country"])
    return "Cobertura de países", not missing, f"sin calendario: {missing or 'ninguno'}"



def check_year_coverage(sales: pd.DataFrame, holidays_dim: pd.DataFrame):
    """V4. Todo año de ventas está cubierto por el calendario."""
    years_sales = set(pd.to_datetime(sales["Order_Date"]).dt.year)
    missing = years_sales - set(holidays_dim["Holiday_Date"].dt.year)
    return "Cobertura de años", not missing, f"sin calendario: {missing or 'ninguno'}"



# ------------------------------------------------------------------------
# Validaciones POSTERIORES al join
# ------------------------------------------------------------------------
def check_row_count(before: pd.DataFrame, after: pd.DataFrame):
    """V5. El join no agrega ni quita filas."""
    return ("Conservación de filas", len(before) == len(after),
            f"antes {len(before)} | después {len(after)}")



def check no row explosion(after: pd.DataFrame):
    """V6. Cada transacción aparece una sola vez (sin multiplicación por cardinalidad)."""
    dups = int(after["Transaction ID"].duplicated().sum())
    return "Sin multiplicación de filas", dups == 0, f"{dups} transacciones repetidas"



def check same transactions(before: pd.DataFrame, after: pd.DataFrame):
    """V7. Mismo conjunto de transacciones (no se pierde ni se inventa ninguna)."""
    same = set(before["Transaction_ID"]) == set(after["Transaction_ID"])
    return "Mismo conjunto de Transaction_ID", same, ""



def check_metrics_preserved(before: pd.DataFrame, after: pd.DataFrame):
    """V8. Los totales de ventas y ganancia no cambian."""
    s0, s1 = before["Sales_Amount"].sum(), after["Sales_Amount"].sum()
    p0, p1 = before["Profit"].sum(), after["Profit"].sum()
    ok = abs(s0 - s1) < 1e-6 and abs(p0 - p1) < 1e-6
    return "Totales conservados", ok, f"Sales {s1:,.2f} | Profit {p1:,.2f}"



def check flag quality(after: pd.DataFrame):
    """V9. Is_Holiday booleana, sin nulos y coherente con Holiday_Name."""
    ok = (
        after["Is Holiday"].dtype == bool
        and after["Is Holiday"].notna().all()
        and (after["Is Holiday"] == after["Holiday Name"].ne("No Holiday")).all()
    )
    return "Is Holiday booleana y coherente", bool(ok), ""



def check_cross_validation(after: pd.DataFrame, holidays_dim: pd.DataFrame):
    """V10. Recalcular el flag con un método independiente (sin merge)."""
```

```
    keys = set(zip(holidays_dim["Country"], holidays_dim["Holiday_Date"]))
    expected = [(c, d) in keys for c, d in
                zip(after["Country"], pd.to_datetime(after["Order Date"]))]
    ok = (after["Is Holiday"].values == pd.Series(expected).values).all()
    return "Verificación cruzada del flag", bool(ok), ""



def check_original_columns(before: pd.DataFrame, after: pd.DataFrame):
    """V11. Las columnas originales no se modificaron."""
    cols = [c for c in before.columns if c not in ("Order_Date", "Country")]
    ok = after[cols].reset_index(drop=True).equals(before[cols].reset_index(drop=True))
    return "Columnas originales intactas", ok, ""



def validate_integration(sales: pd.DataFrame, holidays_df: pd.DataFrame,
                         integrated: pd.DataFrame) -> pd.DataFrame:
    """Ejecuta todas las validaciones, imprime el reporte y devuelve
    un DataFrame con los resultados. Lanza error si alguna falla."""
    holidays_dim = prepare_holidays(holidays_df)
    results = [
        check_sales_key_unique(sales),
        check_holiday_key_unique(holidays_dim),
        check_country_coverage(sales, holidays_dim),
        check_year_coverage(sales, holidays_dim),
        check_row_count(sales, integrated),
        check no row explosion(integrated),
        check same transactions(sales, integrated),
        check_metrics_preserved(sales, integrated),
        check_flag_quality(integrated),
        check cross validation(integrated, holidays dim),
        check original columns(sales, integrated),
    ]

    print("\n[validation] ===== VALIDACIONES DE LA INTEGRACIÓN =====")
    for name, ok, detail in results:
        print(f" [{'OK' if ok else 'FALLA'}] {name}" + (f" -> {detail}" if detail else ""))

    report = pd.DataFrame(results, columns=["validacion", "ok", "detalle"])
    failed = report.loc[~report["ok"], "validacion"].tolist()
    if failed:
        raise ValueError(f"Validaciones fallidas: {failed}")
    print(f" - {len(results)} de {len(results)} validaciones aprobadas")
    return report
```